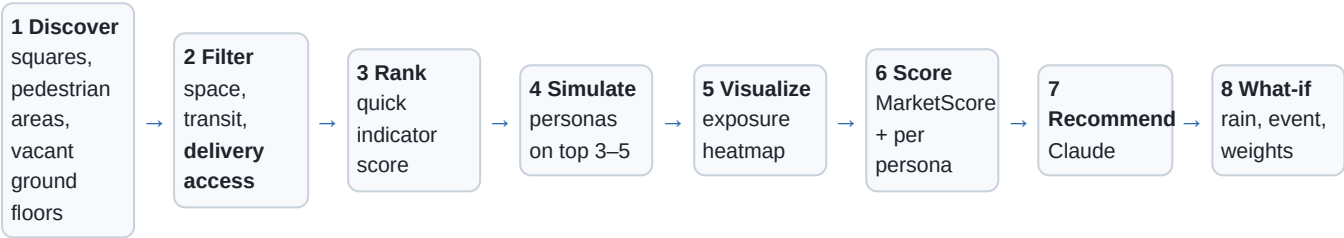


UrbanTwin: Market-Sim

Group implementation plan · Claude Impact Lab #2, Track 2 · Fraunhofer IIS Nürnberg · 7-hour build (13:00–20:00) · 4 people

1. What we ship

A browser decision tool that answers "where should Nuremberg's vegetable market go?". The tool does not start from two fixed sites. It scans the inner city for candidate spaces, filters out the unsuitable ones, ranks the rest, and runs a persona-based pedestrian and delivery simulation on the shortlist. The official candidates (Hauptmarkt, Lorenzkirche, former Kaufhof) always appear as benchmarks. Claude explains the computed result, and a what-if panel recalculates the result under different conditions and priorities.



Two principles shape the design:

- **Deliveries are part of the model.** A site with high footfall that a 3.5-ton van cannot reach at 05:30 fails. Delivery access is a hard filter in step 2 and a score factor in step 6.
- **Stakeholders disagree, and the tool shows it.** The MarketScore is a weighted sum, but the screen always shows the four persona scores beside it. A high total cannot hide a group that loses badly.

The deliverables are:

- the live app,
- a 45–60 second recorded simulation video (the pitch opener and the backup if the network fails),
- a 2-minute pitch.

2. Scope priorities

Priority	Scope
Must ship	About 15 candidates discovered in the Altstadt and inner ring, filter and ranking, persona simulation on the top 3 plus benchmarks, heatmap, MarketScore with adjustable weights, Claude recommendation, one what-if (rainy Saturday)
Should	Christmas-market what-if, weight presets (accessibility first, trader fairness first, local business first), side-by-side compare
Stretch	Citywide discovery beyond the inner ring, live candidate drawing on the map

3. Prior work and what we take from it

Work	What it shows	What we adopt
Park et al., <i>Generative Agents: Interactive Simulacra of Human Behavior</i> (Stanford/Google, arXiv:2304.03442)	LLM agents with profiles, memory, reflection and daily plans produce believable behavior in a sandbox town.	Persona profiles with goals, budgets and daily schedules. We skip memory streams: 100 agents × 15 ticks is too slow and costly for a live demo.
Verma et al., <i>Generative agents in the streets: exploring LLMs in collecting urban perceptions</i> (arXiv:2312.13126)	LLM agents with personas judge street environments from Street View imagery and OSM. The paper is about perception , not route choice.	Persona-specific judgments of a site ("too crowded for my rollator"), written by Claude from the computed metrics
Zheng et al., <i>Urban planning in the era of large language models</i> (MIT Senseable City Lab, Nature Computational Science, 2025)	A conceptualize → generate → evaluate loop in which synthetic residents use competing designs and the designs are scored on equity, travel distance and facility use.	Evaluating competing sites with synthetic residents, using equity and travel distance as criteria (our Fairness and Accessibility)
Zhou et al., <i>LiPUP-MA: a residential experience-centric multi-agent framework for living-in-the-loop participatory urban planning</i> (arXiv:2412.20505), the cyclical "plan, live, judge" line of work	Planner agents propose, resident agents "live" in the plan and complain, and the plan is revised in cycles.	The propose → simulate → judge loop: Claude proposes a mitigation, the engine reruns, and the personas judge the result
Piao et al., <i>AgentSociety</i> (arXiv:2502.08691) and Yan et al., <i>OpenCity</i> (arXiv:2410.21286)	Thousands of LLM agents run in GIS environments to study mobility and policy responses. Scaling them requires batching and grouping agents.	The OpenCity idea of grouping agents into archetypes: one LLM call per persona, not per agent
<i>When Plausible Is Not Realistic: Evaluating Human Mobility in LLM-Based Urban Simulation</i> (arXiv:2606.13835)	LLM-generated mobility can look plausible and still fail to match real movement statistics.	The reason our movement is rule-based and seeded , and footfall is labeled "modeled exposure"

Our design position (hybrid): deterministic Monte Carlo agents decide *where people walk and drive*. This part is fast, reproducible and testable. Claude decides *what it means*. It writes persona judgments, mediates trade-offs, and proposes mitigations that the engine then verifies. This combines the believability of generative agents with the reliability that a planning tool needs, and it puts us next to published work instead of reinventing it.

The "Awesome Urban LLM Agents" list (github.com/usail-hkust/Awesome-Urban-LLM-Agents) collects further work. We could not find a paper titled exactly "Planning, Living, and Judging", so we cite LiPUP-MA for the cyclical approach. Verify the titles before quoting them on slides.

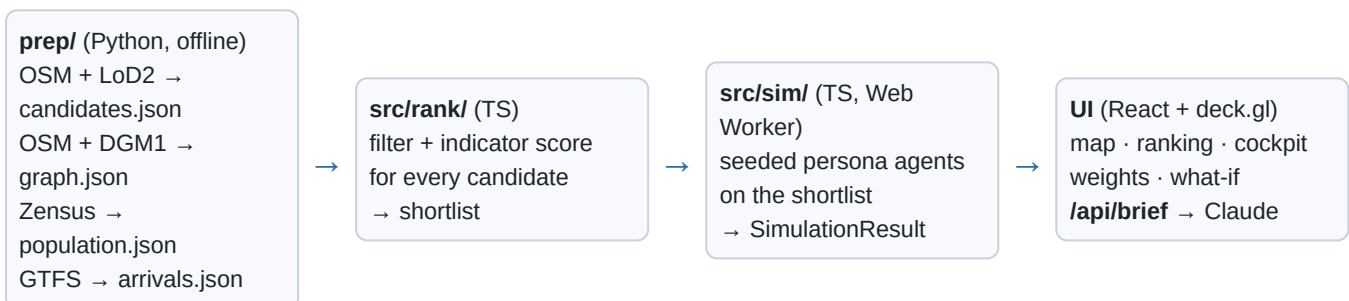
4. Assets we already have

Dataset (in datasets/)	Used in step	Used for
OSM extract, Altstadt (12,889 features)	1, 2, 4	Candidate spaces (place=square, pedestrian areas, large ground floors), the walk and drive graph, 224 bollards, 60 U-Bahn entrances, 34 elevators, shops and attractions

LoD2 CityGML, 4 × 2 km tiles (LDBV)	1, 4, 5	Building footprints and heights: open-space area per candidate, obstacles, the 3D view
DGM1 1 m terrain, 9 tiles	3, 4	Slope per graph edge: senior friction and walkability
DOP20 aerial photo and ALKIS parcels	1, 5	Base layer and visual check of candidate outlines
Zensus 2022 100 m grid, 8,711 cells around Nuremberg: population (624k in the bbox, which includes part of Fürth), share aged 65+, average age	3, 4	Nearby population per candidate, home origins for resident agents, senior share for the persona mix
VGN GTFS timetable	3, 4	Transit frequency per stop and U-Bahn arrival waves for commuter spawning

All sources are open and need no API key. Required credits: "Datenquelle: Bayerische Vermessungsverwaltung – www.geodaten.bayern.de, CC BY 4.0", "© OpenStreetMap contributors", "Statistisches Bundesamt (Destatis), Zensus 2022, dl-de/by-2-0". The former Kaufhof footprint (Königstraße, about 49.4490, 11.0800) must be confirmed against LoD2 before 13:00.

5. Architecture



- **Everything runs in the browser except Claude.** Ranking and simulation are pure, seeded TypeScript functions, so a what-if recalculation takes about a second and works offline.
- **Weights apply after simulation.** The simulation produces raw indicators once per site and scenario. Moving a weight slider only recomputes the weighted sum, so the ranking updates instantly.
- **One small server route** holds the Anthropic key and calls Claude. The key never reaches the browser.
- **Claude explains the numbers and never invents them.** The engine computes every number, and Claude interprets the trade-offs.

6. Data contracts (freeze at 13:20)

```
type Candidate = { id: string; name: string; kind:
"square"|"pedestrian"|"ground_floor"|"benchmark";
  polygon: [lng, lat][]; areaM2: number;
  indicators: { transitScore: number; walkScore: number; population800m: number;
    retailPoi400m: number; attractions400m: number; deliveryAccess: boolean;
vanDistM: number };
  passedFilter: boolean; rejectReason?: string; quickRank?: number };

type Scenario = "SUNNY_SAT" | "RAINY_SAT" | "CHRISTMAS_MARKET";
type Weights = { accessibility: number; footfall: number; fairness: number; localBusiness:
number; walkability: number };

type SimulationResult = { candidateId: string; scenario: Scenario; seed: number;
  criteria: { accessibility: number; footfall: number; fairness: number; localBusiness: number;
walkability: number }; // 0-100 each
  personas: Record<"senior"|"vendor"|"commuter"|"retailer", { score: number; served: number;
droppedOut: number; topFriction: string }>;
  stallExposure: number[]; // visitors passing each stall slot, input to the fairness
Gini
  bottlenecks: { lat: number; lng: number; severity: number; time: string; cause: string }[];
  trips: { persona: string; path: [lng, lat, tSec][] }[]; heat: [lng, lat, weight][] };

type Brief = { recommended: string; why: string[]; comparisons: { site: string; betterAt:
string; worseAt: string }[];
  losers: { persona: string; mitigation: string }[]; councilBriefMd: string };
```

Commit `src/contracts.ts` and fixtures (`mock-candidates.json`, `mock-result-*.json`) at 13:20. After that, nobody waits on anyone.

7. Roles

A · Geodata, Discovery & Ranking

Owns: prep/, public/data/, src/rank/, video recording

- **Discover:** extract about 15–25 candidates from OSM: place=square, pedestrian areas above 800 m², large vacant ground floors. Add the 3 benchmarks.
- **Indicators** per candidate: open area (after subtracting LoD2 buildings), transit frequency from GTFS stops within 300 m, population within 800 m from Zensus, retail and attraction POIs within 400 m, slope
- **Filter:** area below 800 m², no transit stop within 400 m, or no van route to a loading point within 80 m (using vehicle-access edges and bollards) → rejected, with a stated reason
- **Rank:** normalized indicator score, top 3–5 plus benchmarks → shortlist
- graph.json (edges with steps, bollard, slope, vehicleAllowed) and population.json
- After 16:30: the Playwright script that records the video

Done when: the candidate map shows rejected sites greyed out with reasons, the shortlist is defensible, and Lorenzkirche gets flagged for delivery access.

B · Pedestrian & Delivery Twin

Owns: src/sim/ (pure TypeScript, Vitest)

- Seeded RNG and agent origins drawn from real sources: U-Bahn and tram arrivals (GTFS), Zensus residential cells, shopping streets, attractions
- Persona mix: seniors (share taken from the Zensus 65+ grid), commuters, shoppers and tourists, vendor vans at 05:30–07:00
- Dijkstra with per-persona cost: seniors avoid steps, slope over 6% and cobblestones; vans need vehicle edges; rain raises the cost of unsheltered edges
- 30-second ticks: arrive → route → stall visits → leave, with drop-outs when patience or walking budget runs out
- Output: the five criteria, persona scores, stall exposure, bottlenecks, trips, heat
- Scenario switches: rainy day, and a Christmas market that blocks the Hauptmarkt and doubles tourist agents

Done when: one site and scenario runs in under 1.5 s, the same seed gives the same result, and the shortlist sites produce clearly different persona profiles.

C · Map, Ranking & What-if UI

Owns: `src/ui/`

- `deck.gl` over `MapLibre`: DOP20 base, extruded LoD2, candidate polygons colored by rank (rejected ones greyed out with a tooltip reason)
- Simulation view: `TripsLayer` agents by persona, exposure `HeatmapLayer`, bottleneck markers, time slider
- Ranking table and `MarketScore` bars that reorder live as the **five weight sliders** move, plus presets
- What-if bar: Sunny / Rainy / Christmas market
- Persona cards, the Claude brief panel, "Export council brief"

Done when: moving a weight reorders the ranking in under 100 ms and the map stays at 60 fps with 100 trails.

D · Scoring, Claude & Pitch

Owns: `src/score/`, `server/brief.ts`, prompts, the pitch deck

- `score.ts`: normalization, `MarketScore`, fairness Gini, and a minimum-persona guard that flags any persona below 40
- `/api/brief`: sends the ranking, criteria and active weights to `claude-sonnet-5-5` with a forced `write_recommendation` tool (Brief schema). Claude must cite engine numbers and compare against at least two alternatives.
- **Persona judgments**: one Claude call per persona (not per agent) turns that persona's metrics into a first-person verdict for the cards
- **Propose → simulate → judge loop**: Claude proposes a mitigation (a kiosk, a delivery window, a stall layout), B's engine reruns, and the personas judge again
- Re-brief on every what-if, with a short "what changed and why" delta
- Cached briefs per scenario as an offline fallback; key in `.env`
- Integration owner from 17:00; pitch and Method slide

Done when: every number in the brief matches the input, the brief arrives in under 6 s, and the cached fallback works offline.

8. Scoring model

MarketScore = 0.30·Accessibility + 0.25·Footfall + 0.20·Fairness + 0.15·LocalBusiness + 0.10·Walkability
(default weights; adjustable, always normalized to sum to 1)

Criterion (0–100)	Definition	Source
Accessibility	Share of simulated visitors, weighted by persona, who reach the market within their walking budget. Seniors count only if their route has no steps and stays within 200 m of an elevator or stop. Vendor van access is a hard requirement.	Simulation + GTFS
Footfall	Modeled visitor exposure per Saturday. This is a prediction and is not validated against counts.	Simulation
Fairness	Stall-to-stall evenness: $100 \times (1 - \text{Gini of } \text{stallExposure})$. A market where the corner stalls get all the visitors scores low.	Simulation
Local business	Extra footfall past existing shop frontages within 200 m, minus blocked frontage and waste exposure	Simulation + OSM shops

Walkability	Mean route slope, steps, cobblestones and crossing count from visitor origins	Graph + DGM1
-------------	---	--------------

Weight presets for the what-if panel: *Accessibility first* (0.45/0.20/0.15/0.10/0.10), *Trader fairness first* (0.20/0.20/0.40/0.10/0.10), *Local business first* (0.20/0.25/0.10/0.35/0.10).

9. Before the event

Task	Owner
Run prep/: candidates.json, graph.json, population.json, arrivals.json. Commit only the small outputs; datasets/ stays out of git.	A
Confirm the former Kaufhof footprint and loading point from LoD2 and the aerial photo	A
Repo scaffold (Vite, Tailwind, deck.gl, Vitest) with an empty contracts.ts	C
Test that the Anthropic API key works with a one-line tool-call script	D

10. Timeline with gates

Time	A · Discovery & Ranking	B · Twin	C · UI	D · Score & Claude
13:00–13:30 Kickoff	Check prep outputs load	Freeze contracts.ts with D and write the fixtures		Agree the default weights and the filter thresholds
13:30–15:00 Core	Candidate extraction, indicators, filter with reasons	RNG, origins, personas, Dijkstra cost functions, tests	Map, 3D, candidate layer, ranking table on fixtures	score.ts with Gini and guard; /api/brief on fixtures
Gate 15:00: real candidate list with filter reasons, the engine runs one site, the UI shows the fixtures end to end, and Claude returns a valid Brief.				
15:00–16:30 Dynamics	Quick ranking and shortlist, then help B with stall slots and capacities	Stall visits, drop-outs, five criteria, rain and Christmas scenarios	Trips and heatmap, weight sliders, what-if bar	What-if re-brief with deltas, cached fallback, pitch draft
16:30–17:00	Dinner. Run the full pipeline on real data and list the integration bugs.			
17:00–18:00 Integrate	Video recorder, take 1	Calibrate and check that the numbers make sense (the results must follow from the data)	Wire the worker, ranking and brief; polish	End-to-end runs, bug fixes
Gate 18:00: code freeze. The full demo path works three times in a row without a crash.				
18:00–18:30	Final video (captions, credits)	Method slide: definitions and assumptions	Projector contrast	Rehearse the pitch three times
18:30–20:00	Presentations. D presents, C drives the demo, A holds the video fallback, B answers method questions.			

11. Demo script (2 minutes)

1. 0:00, video: the camera flies into the 3D Altstadt. Caption: "The market has to move. Where?"

2. **0:15, Discover and filter:** about 20 candidates appear, and the filter greys out most of them with reasons ("no van access", "too small").
3. **0:35, Simulate:** the shortlist plus the benchmarks run. Agents stream from U-Bahn exits and homes, vans arrive at 05:30, and the heatmap shows exposure per stall.
4. **1:00, Score:** the ranking appears with persona cards. Lorenzkirche wins on footfall and fails on deliveries.
5. **1:15, Claude:** "Site B is recommended because... Site C is better at X but worse at Y." Claude proposes a mitigation for the group that loses. The engine reruns it, and the persona cards update their verdicts.
6. **1:35, What-if:** switch to Rainy, then to "Trader fairness first". The ranking reorders live, and Claude explains what changed.
7. **1:55, Export the council brief.**

12. Risks and fallbacks

Risk	Fallback
Discovery returns odd candidates (parking lots, private courtyards)	Tag-based exclusions plus a manual allow/deny list in <code>sites.json</code>
"The weights decide the winner"	Say it openly: choosing the weights is the planner's value judgment. Show that the winner holds under 2 of 3 presets.
Footfall cannot be validated	Call it "modeled exposure" everywhere; list the assumptions on the Method slide
"Isn't this just generative agents?"	Point to the prior-work table: the movement is rule-based and seeded because LLM mobility is known to be unrealistic, and LLM reasoning is used where it adds value
Venue Wi-Fi or API outage	Cached briefs, all data local, the recorded video
The simulation is late at 15:00	Ranking and Claude run on the quick indicator score alone; A joins B
Merge conflicts	Strict folder ownership, short-lived branches, D merges to main

13. Repository layout

```

claude_imapct_lab/
├─ datasets/          raw downloads (git-ignored), fetch.sh, zensus/fetch_zensus.py, README.md
├─ prep/              A: Python converters → public/data
├─ public/data/       candidates · graph · population · arrivals · mock fixtures
├─ src/contracts.ts   shared types (frozen 13:20)
├─ src/rank/          A: filter + quick ranking
├─ src/sim/           B: pedestrian & delivery twin, worker, tests
├─ src/score/         D: MarketScore, Gini, guards
├─ src/ui/            C: map, ranking, what-if, cockpit
├─ server/brief.ts    D: Claude route, prompts, cached briefs
└─ video/record.ts    A: Playwright capture → urbantwin.mp4

```